

Deep Generative Models

Chapter 6: Generation by Diffusion Process

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

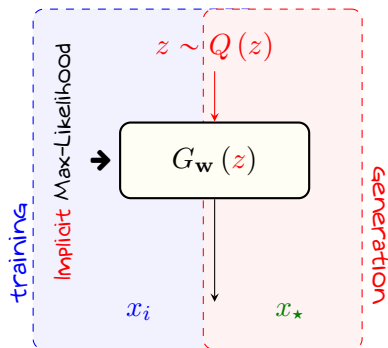
Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

Modern Data Generation

Recall that we are still working with modern generative models

modern generative models mainly learn *how to sample!*



Learning via *Diffusion Process*

Our main goal in this chapter is to

build models that learn by mimicking a diffusion process

- + What is a *diffusion process*?!
 - We understand it after a bit of preliminaries

Langevin Equation

Langevin equation describes a diffusion process in which particle moves under friction and random collisions: the location of the particle in time t satisfies

$$dx(t) = \nabla_x \log P(x(t)) dt + \sqrt{2} dB(t)$$

where $B(t)$ is the Brownian motion process

Brownian Motion

- + What is **Brownian motion** process?
- It's a stochastic process that evolves **i.i.d. Gaussian**

Brownian Motion

The Brownian motion process starts at **zero**, i.e., $B(0) = 0$ and **independently** progresses through time by **Gaussian** distribution

↳ The variance increases **linearly with time**

$$B(t_2) - B(t_1) \sim \mathcal{N}(0, t_2 - t_1)$$

↳ Any difference $B(t_2) - B(t_1)$ is **independent** of past

Langevin Equation: *Learning Viewpoint*

- + *What is important about this equation?*

$$dx(t) = \nabla_x \log P(x(t)) dt + \sqrt{2} dB(t)$$

- It describes the **physical diffusion**

Diffusion Process by Langevin Equation

As time evolves, the location of the particle at time t , i.e., $x(t)$, is **random** and **its distribution** evolves to $P(x)$ through time

- + *But how can we use such a fact?!*
- Well! Let's be a bit **naïve** and **approximate** solution of this equation

Langevin Dynamics: Markov Chain

Say we move **tiny steps** in time: we *shift step by step* from t to $t + \delta$ and approximate $dt \approx \delta$ as well as

$$dx(t) \approx x(t + \delta) - x(t) \equiv x^{(i+1)} - x^{(i)}$$

$$dB(t) \approx B(t + \delta) - B(t) \equiv \Delta B^{(i)} \sim \mathcal{N}(0, \delta)$$

Now, we could say

$$x^{(i+1)} - x^{(i)} = \nabla_x \log P(x^{(i)}) \delta + \sqrt{2} \Delta B^{(i)}$$

We can say $\Delta B^{(i)} = \sqrt{\delta} \varepsilon^{(i)}$ with $\varepsilon^{(i)} \sim \mathcal{N}(0, 1)$ and write

$$x^{(i+1)} = x^{(i)} + \delta \nabla_x \log P(x^{(i)}) + \sqrt{2\delta} \varepsilon^{(i)}$$

The **Markov process** $x_1 \rightarrow x_2 \rightarrow \dots$ is what we call **Langevin dynamics**

Key Property of Langevin Dynamics

- + *Well! Then what?!*
- We can use this process to **sample** $P(x)$

Sampling via Langevin Dynamics

Recall that in Langevin equation $x(t)$ **converges** in **distribution** to $P(x)$

- 1 Start by an arbitrary point $x^{(0)}$
- 2 Evolve through time by

$$x^{(i+1)} = x^{(i)} + \delta \nabla_x \log P(x^{(i)}) + \sqrt{2\delta} \varepsilon^{(i)}$$

- 3 Get a sample of $P(x)$

Recall: Sampling EBM

- + Wait a moment! This sounds *familiar!*
- Yep! We used it to *sample EBM*

Langevin_Sampling() :

- 1: *Initiate sample $x^{(0)}$ and choose a converging series $\{\epsilon_t\}$*
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Sample $\eta_t \sim \mathcal{N}(0, 1)$
- 4: Update $x^{(t)} \leftarrow x^{(t-1)} + \frac{\epsilon_t}{2} \nabla_x \log P_{\mathbf{w}}(x^{(t-1)}) + \sqrt{\epsilon_t} \eta_t$
- 5: **end for**
- 6: **return** $x^{(T)}$ *for T larger than *burn-in period**

Alternative Objective for Generation

This brings up this idea:

Why don't we *model* and *learn* directly the term $\nabla_x \log P(x)$?!

If we learn $\nabla_x \log P(x) \rightsquigarrow$ we can sample $P(x)$ by *mimicking diffusion*

- + Why should this be *better* than learning $P(x)$ itself?!
- Well! There are two key reasons
 - ① As we have seen in Chapter 3, this can be *computationally easier*

$$\nabla_x \log P(x) = \nabla_x \log \frac{\exp\{-\mathcal{E}(x)\}}{\mathcal{Z}} = -\nabla_x \mathcal{E}(x)$$

- ② It opens a *way* to learn the distribution *implicitly*
 - ↳ We will see this in this chapter

Score Function

Score Function

The **score function** is defined as the gradient of the log-likelihood **w.r.t. argument**

$$s(x) = \nabla_x \log P(x)$$

! Attention: Score Function vs Informant

The **score function** is different from the **informant** which is also called **score** sometimes in statistics: if we have a model $P_{\mathbf{w}}(x)$; then, the informant is

$$\iota_{\mathbf{w}}(x) = \nabla_{\mathbf{w}} \log P_{\mathbf{w}}(x)$$

We just keep calling this informant to avoid confusion

Score Function: Gaussian Example

Example: Say we have a **Gaussian** distribution, i.e.,

$$P(x) = \frac{\exp\{-x^2/2\}}{\sqrt{2\pi}}$$

The **score function** in this case is

$$s(x) = \frac{d}{dx} \log P(x) = -\frac{1}{2} \frac{dx^2}{dx} = -x$$

We note that in this example

- The score function is a **random variable**
- The mean and variance are

$$\mathbb{E}_{x \sim P} \{s(x)\} = 0$$

$$\mathbb{E}_{x \sim P} \{s^2(x)\} = 1$$

Properties of Score Function: *Normalization Independence*

Normalization Independence

The score function does **not** change by any distribution **scaling**

- + How can such property help?
- When we know the distribution **up to a constant**

Say we know joint distribution $P(x, y)$ and look for score function of $P(x|y)$

- Computing $P(x|y)$ itself **needs marginalization over x**
- The score function is though given by

$$\nabla_x \log P(x|y) = \nabla_x \log \frac{P(x, y)}{P(y)} = \nabla_x \log P(x, y)$$

which **does not need any marginalization**

Properties of Score Function: Zero-Mean

Score Function is Zero-Mean

For well-defined distributions, the *expected score function* is *zero*, i.e.,

$$\mathbb{E}_{x \sim P} \{s(x)\} = 0$$

Let's expand the expression

$$\begin{aligned} \mathbb{E}_{x \sim P} \{s(x)\} &= \int s(x) P(x) dx = \int \nabla_x \log P(x) P(x) dx \\ &= \int \frac{\nabla_x P(x)}{P(x)} P(x) dx = \int \nabla_x P(x) dx = 0 \end{aligned}$$

Say we want to compute entry i

$$\int \frac{\partial}{\partial x_i} P(x) dx = P(x_i) \Big|_{-\infty}^{+\infty} = 0 \leftarrow \text{if } P \text{ is decaying}$$

Properties of Score Function

Score Function is Zero-Mean

For well-defined distributions, the covariance matrix is the negative expected Jacobian of the score function, i.e.,

$$\text{Cov} [s(x)] = -\mathbb{E}_{x \sim P} \{ \nabla_x s(x) \}$$

We can again show this by definition

$$\begin{aligned} \text{Cov} [s(x)] &= \mathbb{E}_{x \sim P} \left\{ (s(x) - \mathbb{E}_{x \sim P} \{s(x)\}) (s(x) - \mathbb{E}_{x \sim P} \{s(x)\})^\top \right\} \\ &= \mathbb{E}_{x \sim P} \left\{ s(x) s(x)^\top \right\} = \dots = -\mathbb{E}_{x \sim P} \{ \nabla_x s(x) \} \end{aligned}$$

Score Models

Back to our problem: *our idea is to learn the **score function** of data*

Score-based Model

Score-based model $s_{\mathbf{w}}(\cdot) : \mathbb{R}^d \mapsto \mathbb{R}^d$ is a computational model approximating the score function $s(x) = \nabla_x \log P(x)$ of the data from its sample

The key point about the score-based models is that

- They **do not need to fit into a specific definition**
 - ↳ Models for distributions should add up to 1
- They could be **directly used for sampling**
 - ↳ We just need to replace them in **Langevin dynamics**

Training Score Models

- + Say we choose a NN as score model, how can we **train** it?!
- We try to do **score matching**!

Score Matching

We train the **model**, such that its difference to the **true score** is minimized. This means that we train the model for the **risk function**

$$R(\mathbf{w}) = \mathbb{E}_{x \sim P} \{ \|s_{\mathbf{w}}(x) - s(x)\|^2 \}$$

- + Excellent! We have **neither the true score nor data distribution**!
- So was it the case when we wanted to **minimize KL divergence**
 - ↳ We ended up with **maximum likelihood**!

Training Score: Score Risk

Let's try to look at this risk function: *given the definition we have*

$$\begin{aligned} R(\mathbf{w}) &= \mathbb{E}_{x \sim P} \left\{ \|s_{\mathbf{w}}(x) - s(x)\|^2 \right\} \\ &= \mathbb{E}_{x \sim P} \left\{ \|s_{\mathbf{w}}(x)\|^2 \right\} + \mathbb{E}_{x \sim P} \left\{ \|s(x)\|^2 \right\} - 2\mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^\top s(x) \right\} \end{aligned}$$

Since we want to minimize *w.r.t. model* parameters $\mathbf{w}$, we can say

$$\underset{\mathbf{w}}{\operatorname{argmin}} R(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmin}} \left[\mathbb{E}_{x \sim P} \left\{ \|s_{\mathbf{w}}(x)\|^2 \right\} - 2\mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^\top s(x) \right\} \right]$$

Looking at the above term, we could say

- ✓ The term $\mathbb{E}_{x \sim P} \left\{ \|s_{\mathbf{w}}(x)\|^2 \right\}$ *can be estimated by sampling*
- ✗ The term $\mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^\top s(x) \right\}$ *still needs data distribution*

Training Score: Score Divergence

Let's try to further expand the latter term

$$\begin{aligned}
 \mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^{\top} s(x) \right\} &= \int s_{\mathbf{w}}(x)^{\top} s(x) P(x) dx \\
 &= \int s_{\mathbf{w}}(x)^{\top} \underbrace{\nabla_x \log P(x) P(x)}_{\nabla_x P(x)} dx \\
 &= \int s_{\mathbf{w}}(x)^{\top} \nabla_x P(x) dx
 \end{aligned}$$

Using the definition of the gradient, we can write

$$\begin{aligned}
 \int s_{\mathbf{w}}(x)^{\top} \nabla_x P(x) dx &= \int \sum_i s_{\mathbf{w},i}(x) \frac{\partial}{\partial x_i} P(x) dx \\
 &= \sum_i \int s_{\mathbf{w},i}(x) \frac{\partial}{\partial x_i} P(x) dx
 \end{aligned}$$

Recall: *Integration by Part*

We want to compute

$$\int_a^b f(x) \frac{d}{dx} g(x) dx$$

But, we would like to work with *g itself* instead of *its derivative*

We use the fact that

$$\frac{d}{dx} [f(x) g(x)] = \frac{d}{dx} f(x) g(x) + f(x) \frac{d}{dx} g(x)$$

and write the original integral as

$$\int_a^b f(x) \frac{d}{dx} g(x) dx = f(x) g(x) \Big|_a^b - \int_a^b \frac{d}{dx} f(x) g(x) dx$$

Training Score: Score Divergence

Using integration by part, we could say

$$\begin{aligned}\int s_{\mathbf{w},i}(x) \frac{\partial}{\partial x_i} P(x) dx &= \textcolor{red}{s}_{\mathbf{w},i}(x) P(x) \Big|_{-\infty}^{\infty} - \int \frac{\partial}{\partial x_i} s_{\mathbf{w},i}(x) P(x) dx \\ &= \textcolor{red}{0} - \mathbb{E}_{x \sim P} \left\{ \frac{\partial}{\partial x_i} s_{\mathbf{w},i}(x) \right\}\end{aligned}$$

This concludes that the cross-product term is given by

$$\begin{aligned}\mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^{\top} s(x) \right\} &= \sum_i \int s_{\mathbf{w},i}(x) \frac{\partial}{\partial x_i} P(x) dx \\ &= - \sum_i \mathbb{E}_{x \sim P} \left\{ \frac{\partial}{\partial x_i} s_{\mathbf{w},i}(x) \right\} \\ &= - \mathbb{E}_{x \sim P} \left\{ \sum_i \frac{\partial}{\partial x_i} s_{\mathbf{w},i}(x) \right\}\end{aligned}$$

Training Score: Score Divergence

Score Divergence

The divergence of the score-model $s_{\mathbf{w}}(\cdot)$ is defined as

$$\operatorname{div}(s_{\mathbf{w}}(x)) = \sum_i \frac{\partial}{\partial x_i} s_{\mathbf{w},i}(x) = \operatorname{tr}\{\nabla_x s_{\mathbf{w}}(x)\}$$

Using this definition, we could say

$$\mathbb{E}_{x \sim P} \left\{ s_{\mathbf{w}}(x)^{\top} s(x) \right\} = -\mathbb{E}_{x \sim P} \left\{ \operatorname{div}(s_{\mathbf{w}}(x)) \right\}$$

which can be *estimated from data samples!*

Training Score: *Hyvärinen Result*

Aapo Hyvärinen was the one who showed this result

Score Risk

The score matching is equivalent to minimizing

$$R_{\text{score}}(\mathbf{w}) = \mathbb{E}_{x \sim P} \{ \|s_{\mathbf{w}}(x)\|^2 \} - 2\mathbb{E}_{x \sim P} \{ \text{div}(s_{\mathbf{w}}(x)) \}$$

*This means that the **risk** for **score matching** can be **estimated** as*

$$\hat{R}_{\text{score}}(\mathbf{w}) = \hat{\mathbb{E}}_{x \sim P} \{ \|s_{\mathbf{w}}(x)\|^2 \} - 2\hat{\mathbb{E}}_{x \sim P} \{ \text{div}(s_{\mathbf{w}}(x)) \}$$

*using the **data samples***

Training Computational Score Models

Score_Train($\mathbb{D}$:dataset):

```

1: Initiate the score model  $s_{\mathbf{w}}$  with some  $\mathbf{w}$ 
2: for multiple epochs do
3:   Sample a batch of data samples  $\{x^j : j = 1, \dots, n\}$  from  $\mathbb{D}$ 
4:   for sample  $j = 1, \dots, n$  do
5:     Compute the model Jacobian  $\mathbf{J}^j \leftarrow \nabla_x s_{\mathbf{w}}(x^j)$ 
6:     Compute sample risk as  $R^j \leftarrow \|s_{\mathbf{w}}(x^j)\|^2 + \text{tr}\{\mathbf{J}^j\}$ 
7:     Backpropagate to compute sample gradient  $\nabla R^j$ 
8:   end for
9:   Update  $\mathbf{w}$  using  $\text{Opt\_avg} \left\{ \nabla_{\mathbf{w}} \hat{R}^j \right\}$ 
10: end for
11: return trained score model

```

This is though computationally *expensive*

↳ We need to *backpropagate* on $\nabla_x s_{\mathbf{w}}(x)$

Sampling Computational Score Models

Score_Sampling():

- 1: *Initiate sample $x^{(0)}$ and choose a converging series $\{\epsilon_t\}$*
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Sample $\eta_t \sim \mathcal{N}(0, 1)$
- 4: Update $x^{(t)} \leftarrow x^{(t-1)} + \frac{\epsilon_t}{2} s_{\mathbf{w}} \left(x^{(t-1)} \right) + \sqrt{\epsilon_t} \eta_t$
- 5: **end for**
- 6: **return** $x^{(T)}$ *for T larger than **burn-in period***